Day 93: Sampling Strategies From Scratch — Greedy, Temperature, Top-k, Top-p

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 14, Day 3 | Complete study guide

Watch: Temperature, Top-K & Top-P Explained in 10 Minutes (bababoss) | Nucleus Sampling: The Curious Case of Neural Text Degeneration (TechViz)

Script: `day93_sampling_strategies.py` · every number in this guide is copied by program from `day93_run_output.txt`, the real output of that script.

The same trained model writes very different text depending on how each next character is chosen. Measured on 20 held-out prompts with Day 92's mini-GPT: greedy decoding loops (only 0.059 of its word pairs are distinct, against 0.909 in the real text); raising the temperature trades safe, repetitive text for varied text with more non-words (0.806 real words at 1.2). Top-k 5 and top-p 0.9 sit in between, close to temperature 0.8. No strategy matches the real text on every measure at once.

0. Where today fits

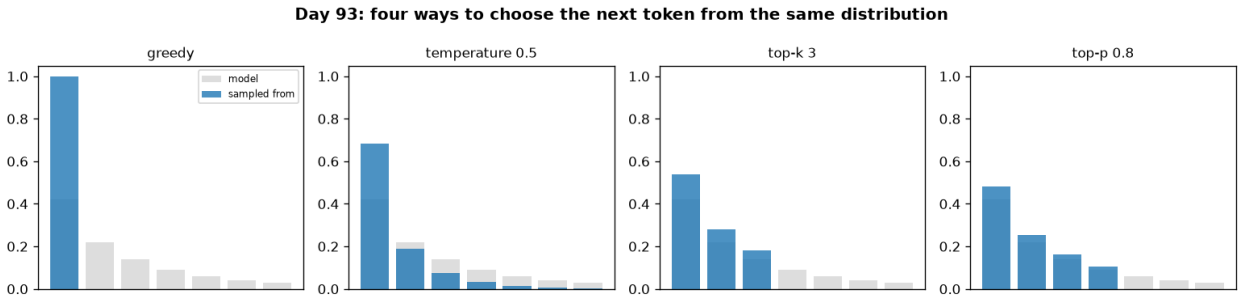
DEFINITION — start here, no prior knowledge assumed

A language model gives a probability to every possible next token. A **sampling strategy** decides which one to write. **Greedy** always takes the most probable. **Temperature** t divides the scores before the softmax: below 1 it sharpens the distribution, above 1 it flattens it. **Top-k** keeps the k most probable tokens, **top-p** (nucleus) the smallest set whose probabilities add up to p; both then renormalise and draw.

REAL-WORLD SCENARIO

Choosing what to order at a restaurant: always the favourite (greedy), anything on the menu at random (temperature 1), or only from the three or four dishes you like (top-k, top-p).

SYSTEM DIAGRAM



The same next-token distribution cut four ways.

Today's part for Day 97

`day93_exercise.py` is the sampler on its own, checked against exact distributions and at its two limits (very low temperature behaves like greedy, very high like a uniform draw).

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Do the samplers draw what they claim?

DEFINITION — start here, no prior knowledge assumed

A toy distribution over five tokens; 200,000 draws per sampler; the observed frequencies against the exact distribution each sampler should draw from.

REAL-WORLD SCENARIO

Rolling a die 200,000 times to check it is fair before using it in a game.

MATHEMATICS

temperature: $q_i = exp(z_i/t) / \Sigma\, exp(z_j/t) = p_i^{1/t} / \Sigma\, p_j^{1/t}$

top-k: $q_i \propto p_i$ *for the k largest;* *top-p:* $q_i \propto p_i$ *for the smallest set with* $\Sigma\, p \geq p_0$*;* *then draw by inverse CDF*

IMPLEMENTATION

```
def sample_next(logits, rng, temperature=1.0, top_k=None, top_p=None):
    if temperature == 0:
        return logits.argmax(axis=-1)
    z = logits / temperature
    p = np.exp(z - z.max(axis=-1, keepdims=True)); p /= p.sum(axis=-1, keepdims=True)
    if top_k is not None:
        kth = np.sort(p, axis=-1)[:, -top_k][:, None]
        p = np.where(p >= kth, p, 0.0)
    if top_p is not None:
        order = np.argsort(-p, axis=-1)
        sp = np.take_along_axis(p, order, axis=-1)
        keep_sorted = np.cumsum(sp, axis=-1) - sp < top_p
        keep = np.zeros_like(p, dtype=bool); np.put_along_axis(keep, order, keep_sorted, axis=-1)
        p = np.where(keep, p, 0.0)
    p /= p.sum(axis=-1, keepdims=True)
    u = rng.rand(len(p), 1)
    return np.minimum((np.cumsum(p, axis=-1) < u).sum(axis=-1), p.shape[-1] - 1)
```

ACTUAL OUTPUT

```
probabilities: [0.5, 0.2, 0.15, 0.1, 0.05]
sampler          expected                                  drawn                                     largest gap
temperature 1.0  [0.5, 0.2, 0.15, 0.1, 0.05]               [0.498, 0.2, 0.151, 0.1, 0.05]                 0.0019
temperature 0.5  [0.769, 0.123, 0.069, 0.031, 0.008]       [0.771, 0.123, 0.068, 0.03, 0.008]             0.0016
top-k 2          [0.714, 0.286, 0.0, 0.0, 0.0]             [0.714, 0.286, 0.0, 0.0, 0.0]                  0.0007
top-p 0.8        [0.588, 0.235, 0.176, 0.0, 0.0]           [0.587, 0.235, 0.178, 0.0, 0.0]                0.0016
greedy: [0, 0, 0] (always token 0)
(with 200,000 draws, random gaps of a few thousandths are expected)
Actual output from this lesson's run.
```

What it means

Every sampler draws from the distribution it claims, within 0.0019 at most over 200,000 draws, the size of random error expected. Top-p 0.8 keeps three tokens (0.5 + 0.2 = 0.7 is still below 0.8, so the third is needed), not two.

WHY CHOOSE THIS?

One function, four strategies, checked exactly.

WHY NOT THIS? (TRADEOFFS)

Checks the mechanics, not whether the text is good.

2. Seven strategies, 20 prompts, measured

DEFINITION — start here, no prior knowledge assumed

Day 92's model continues 20 prompts of 32 held-out characters by 400 characters with each strategy. Four measures: distinct word pairs (repetition), real words (share found in the training text), model bits (how surprising the model finds its own text: low = safe) and judge bits (how surprising Day 91's Kneser-Ney model, trained only on the training text, finds it: a rough 'is this Shakespeare-like' score).

REAL-WORLD SCENARIO

Seven drivers on the same route: who goes in circles, who takes wild detours.

SPECIFICATION

Model	Day 92's big mini-GPT (824,129 weights), loaded from day92_mini_gpt.npz.
Prompts	20 random places in the held-out text; the real continuation is the reference row.
Seed	RandomState(0) for every strategy, so they differ only in how they choose.

Why this test

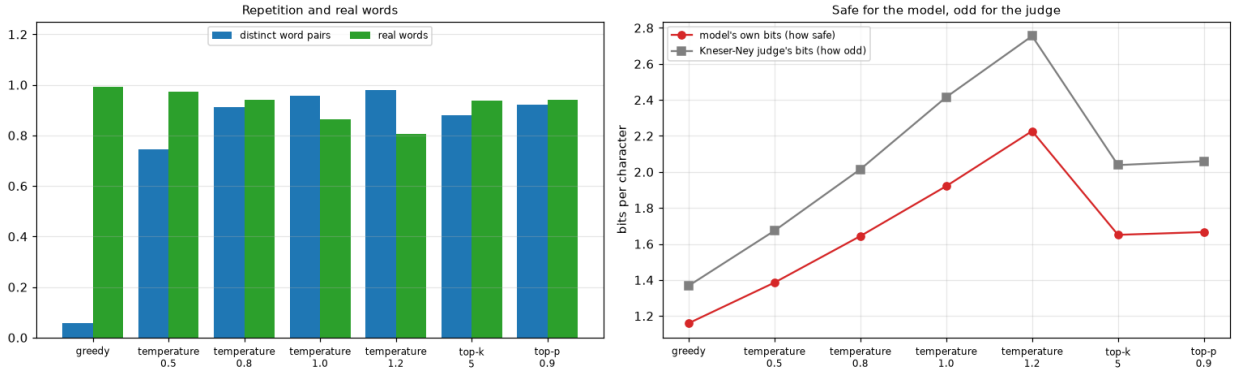
Reading a few samples is the usual way to judge a sampler, and it is easy to fool oneself. Four numbers on 20 prompts compare all seven at once.

IMPLEMENTATION

```
def generate(P, cfg, prompts, n, rng, **how):
    ids = [list(p) for p in prompts]
    for _ in range(n):
        x = np.array([s[-cfg["T"]:] for s in ids])
        lg = d90.gpt_forward(x, P, cfg["h"])[0][:, -1]
        for s, t in zip(ids, sample_next(lg, rng, **how)):
            s.append(int(t))
    return [s[len(p):] for s, p in zip(ids, prompts)]
```

ACTUAL OUTPUT

```
strategy          distinct pairs  real words  model bits  judge bits
(real text)                0.909       0.933       2.226       2.208
greedy                     0.059       0.993       1.160       1.368
temperature 0.5            0.744       0.974       1.385       1.673
temperature 0.8            0.912       0.940       1.644       2.015
temperature 1.0            0.959       0.864       1.921       2.415
temperature 1.2            0.980       0.806       2.227       2.755
top-k 5                    0.881       0.936       1.651       2.038
top-p 0.9                  0.921       0.940       1.666       2.059
Actual output from this lesson's run.
```



Left: distinct word pairs and real words per strategy. Right: how surprising the text is to the model itself and to the Kneser-Ney judge.

What it means

Greedy gives the text the model finds least surprising (1.160 bits) and it is degenerate: almost every word pair repeats (0.059 distinct). **Temperature** moves smoothly between the extremes: from 0.5 to 1.2 distinct pairs rise (0.744 to 0.980) while real words fall (0.974 to 0.806) and the judge finds the text odder (1.673 to 2.755). The real text sits at 0.909 distinct pairs, 0.933 real words and 2.208 judge bits. **Top-k 5** and **top-p 0.9** land near temperature 0.8 (0.881 and 0.921 distinct pairs, 0.936 and 0.940 real words). Temperature 0.8 and top-p 0.9 come closest to the real text's repetition and real-word rate; every truncated or cooled strategy is less surprising to the judge than real text, so all of them write safer text than Shakespeare.

WHY CHOOSE THIS?

Four measures that show the trade-off instead of hiding it.

WHY NOT THIS? (TRADEOFFS)

20 prompts, one seed; none of the four measures captures sense.

3. One prompt, every strategy

DEFINITION — start here, no prior knowledge assumed

The first 160 characters of each continuation of the first prompt, the real text first.

REAL-WORLD SCENARIO

Hearing seven readers finish the same sentence.

ACTUAL OUTPUT

```
prompt: 'her, so it is.\n\nPETRUCHIO:\nCome,'
--- (real text) ---
 go along, and see the truth hereof;
```

```
For our first merriment hath made thee jealous.
HORTENSIO:
Well, Petruchio, this has put me in heart.
Have to my widow! an
--- greedy ---
 the prince of the prince of the seat,
And the seal of the season of the seat of the sea
That the seal of the seal of the season of the country,
And the seal of
--- temperature 0.5 ---
 you shall be excused, and I cannot say
And the duke of the friends of a wars,
To be the sounder of your father with a crown
That I should be in all this shore
--- temperature 0.8 ---
 you should seem of rule in
That he scond it to crown! I am grace:
For I was my temptress spite, my name for your house!
BIANCA:
Bastard he that you his short
--- temperature 1.0 ---
 you sister, already! What 'sagged it
seal with cross'd never: but but I was love
Doth night. What womb thou wast much but
O'er. I am see hence say forget him,
--- temperature 1.2 ---
 you rise is't up, thy fake
That he say God, that made himselves, marry long.
If I mistress, with aspared to templess
Have have placed and blood, when you,
Weak
--- top-k 5 ---
 that to her, as he, what i' the people
And to bed.
POLIXENES:
In my father's succession.
PRINA:
What is thy love, I am a breath and heard thee
Whereof thy ha
--- top-p 0.9 ---
 you should seem me, you have counter we
into the name of winted to speaks it out blow:
For thou mays of his tent one have have point.
KING RICHARD III:
Thou a
Actual output from this lesson's run.
```

What it means

The numbers in Section 2 read as text: greedy circles ('the seal of the season of the seat'), temperature 0.5 is fluent but bland, temperature 1.2 invents words, and top-k and top-p write speaker names and verse lines. None of them follows the scene the prompt started.

WHY CHOOSE THIS?

Makes the measures concrete.

WHY NOT THIS? (TRADEOFFS)

One prompt is an anecdote; Section 2 is the evidence.

Interview preparation — technical questions

Q: What does temperature do to the distribution?

A: Raises each probability to the power 1/t and renormalises: t < 1 sharpens, t > 1 flattens.

Q: How is top-p different from top-k?

A: Top-p keeps a varying number of tokens: as many as it takes to reach mass p.

Q: How do you check a sampler?

A: Many draws against the exact distribution; here within 0.0019 over 200,000 draws.

Q: How is sampling done in code?

A: Inverse CDF: one uniform number per row; the first token whose cumulative probability reaches it.

Q: Why is greedy decoding repetitive?

A: It always takes the likeliest token, and likely continuations of a loop are the loop: 0.059 distinct pairs here.

Interview preparation — theoretical questions

Q: Why is the most likely text not the best text?

A: Real text is not the most likely text; greedy's 1.160 bits is far below the real text's 2.226.

Q: What does top-p protect against?

A: The long tail of unlikely tokens, which at temperature 1 produces non-words (0.864 real words).

Q: Is there a best strategy?

A: Not on every measure: temperature 0.8 and top-p 0.9 come closest here, but both are safer than real text.

Q: Why use an outside judge?

A: The model's own bits only say how typical the text is for the model, not for the language.

Key takeaway and what's next

> **KEY TAKEAWAY**
> How the next token is chosen changes the same model's text a great deal. Greedy loops, high temperature invents words, and truncation (top-k, top-p) or a moderate temperature gives varied text with real words. All of them write safer text than the real Shakespeare, by both the model's and an outside judge's measure.

- Samplers checked: within 0.0019 of the exact distributions over 200,000 draws.
- Greedy: 0.059 distinct word pairs (real text 0.909).
- Temperature 0.5 to 1.2: distinct pairs 0.744 to 0.980; real words 0.974 to 0.806.
- Top-k 5, top-p 0.9: 0.881 and 0.921 distinct pairs, 0.936 and 0.940 real words.

Suggested watch order, if starting from zero

- 1. Temperature, Top-K & Top-P Explained in 10 Minutes (bababoss) — the four strategies, quickly.
- 2. Nucleus Sampling: The Curious Case of Neural Text Degeneration (TechViz) — why maximisation degenerates, and nucleus sampling.

Day 94 preview

- **Judging the text properly.** 120 passages, a classifier that tries to tell them from real Shakespeare, a test for copying, and the play's shape.